ACADEMIC PROJECT REPORT

Library Management System

A full-stack web application built with ASP.NET Core 8 MVC & SQL Server

GITHUB REPOSITORY

github.com/Kavyansh180/Library-Management-System

FRAMEWORK	LANGUAGE	DATABASE	ORM	YEAR
ASP.NET Core 8	C# .NET 8.0	SQL Server Express	Entity Framework Core 8	2026

Table of Contents

54	8	6	12	2
FILES COMMITTED	CONTROLLERS	DB TABLES	VIEWS CREATED	DATABASES

✅ The project is fully functional, version-controlled and deployed locally at **http://localhost:5000**

1. Project Overview & Objectives

The **Library Management System (LMS)** is a web-based application developed using the ASP.NET Core 8 MVC framework. It provides a comprehensive solution for managing library resources including books, students, librarians, borrowing records, and publications such as newspapers and magazines.

The system was developed to digitize and automate the day-to-day operations of a library, replacing manual record-keeping with a fast, reliable, and user-friendly interface accessible from any modern web browser.

Key Objectives

Book Management

Add, edit, delete and search books with availability tracking.

Student Records

Maintain student profiles with search and pagination support.

Librarian Management

Full CRUD operations for librarian staff records.

Borrow & Return

Issue books to students and track return dates.

Publications

Manage newspapers and magazines separately from books.

Dashboard

Real-time summary of all library statistics in one view.

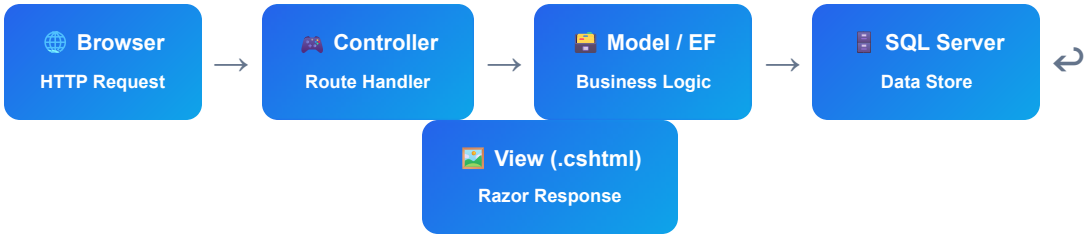
2. Technology Stack

Layer	Technology	Version	Purpose
Backend Framework	ASP.NET Core MVC	8.0	Web application framework
Programming Language	C#	.NET 8.0	Application logic

Layer	Technology	Version	Purpose
ORM	Entity Framework Core	8.0.0	Database access for Books, BorrowRecords, Publications
Authentication	ASP.NET Core Identity	8.0.0	User management & identity tables
Database	SQL Server Express	2017	Primary data store
Raw SQL	Microsoft.Data.SqlClient	5.1.1	Students & Librarians queries
Frontend CSS	Bootstrap	5.3.0	Responsive UI components
Frontend JS	jQuery	3.6.0	Client-side interactivity
Version Control	Git / GitHub	—	Source code management

3. System Architecture

The application follows the **Model-View-Controller (MVC)** architectural pattern, separating concerns clearly across three layers:



Request Flow

A browser request hits the Kestrel web server → Routing middleware maps it to the correct controller action → The action fetches data from the database via EF Core or raw SQL → A Razor view renders HTML → Response sent back to browser.

Dual Database Strategy

LMS database stores Books, BorrowRecords, Publications, Students and Librarians. **IdentityCoreDB** stores ASP.NET Identity tables (AspNetUsers, AspNetRoles, etc.). Both connect to SQL Server Express via `appsettings.json`.

4. Project Structure

```
LMSystem/
├── Controllers/ ← 8 MVC controllers (request handlers)
│   ├── BooksController.cs
│   ├── BorrowController.cs
│   ├── DashboardController.cs
│   ├── HomeController.cs
│   ├── LibrarianController.cs
│   ├── LoginController.cs
│   ├── PublicationsController.cs
│   └── StudentController.cs
├── Data/ ← EF Core DbContext classes
│   ├── ApplicationDbContext.cs ← ASP.NET Identity
│   └── LibraryContext.cs ← Books, BorrowRecords, Publications
├── Migrations/ ← EF Core migration files
│   ├── Identity/
│   └── Library/
├── Models/ ← 7 domain model classes
│   ├── Book.cs
│   BorrowRecord.cs
│   DashboardModel.cs
│   ├── LibrarianModel.cs
│   LoginModel.cs
│   ├── Publication.cs
│   StudentModel.cs
├── ViewModels/ ← 5 view-specific models
├── Views/ ← 17 Razor (.cshtml) views
│   ├── Books/Home/Login/Dashboard/Borrow/
│   ├── Librarian/Student/Publications/
│   └── Shared/ (_Layout, Error, NotFound …)
├── wwwroot/ ← Static files (CSS, JS, images)
├── Program.cs ← App bootstrap & DI configuration
└── appsettings.json ← Connection strings & logging
```

5. Database Design

The project uses **two SQL Server Express databases** managed via Entity Framework Core migrations and raw SQL commands.

Database 1: LMS (Library Data)

Table	Column	Type	Description
Books13	BookId (PK)	INT IDENTITY	Primary key
	Title	NVARCHAR	Book title
	Author	NVARCHAR	Author name
	ISBN	NVARCHAR	ISBN number
	PublishedDate	DATETIME2	Publication date
	IsAvailable	BIT	Availability flag (true/false)
BorrowRecords13	BorrowRecordId (PK)	INT IDENTITY	Primary key
	BookId (FK)	INT	References Books13
	BorrowerName	NVARCHAR	Student name
	BorrowerEmail	NVARCHAR	Student email
	BorrowDate	DATETIME2	Date borrowed
	ReturnDate	DATETIME2?	Date returned (nullable)
Publications	Id (PK)	INT IDENTITY	Primary key
	Title	NVARCHAR	Publication title
	Publisher	NVARCHAR	Publisher name
	PublishedDate	DATETIME2	Date published
	Type	INT (Enum)	0 = Newspaper, 1 = Magazine

Table	Column	Type	Description
Students	StudentId (PK)	INT IDENTITY	Primary key
	Name	NVARCHAR(100)	Student name
	Age	INT	Student age
	Phone	NVARCHAR(20)	Contact number
Librarians	LibrarianId (PK)	INT IDENTITY	Primary key
	Name	NVARCHAR(100)	Librarian name
	Age	INT	Age
	Phone	NVARCHAR(20)	Contact phone

Database 2: IdentityCoreDB (ASP.NET Identity)

AspNetUsers

User accounts with hashed passwords

AspNetRoles

Role definitions (e.g. Admin)

AspNetUserRoles

User ↔ Role mapping

AspNetUserTokens

Auth tokens & refresh tokens

6. Modules & Features

Module 1 — Book Management `BOOKSCONTROLLER`

Full CRUD operations for managing library books. Each book tracks its availability status. When a book is borrowed its `IsAvailable` flag is set to `false` and restored on return. Features include paginated listing, search, add/edit/delete with EF Core and antiforgery token protection.

Module 2 — Borrow & Return System `BORROWCONTROLLER`

Students can borrow available books by providing their name, email and phone. The system validates availability before issuing. On return, the `ReturnDate` is recorded and the book is marked available again. Both operations use EF Core transactions.

Module 3 — Student Management `STUDENTCONTROLLER`

Manages enrolled student records using raw SQL via `SqlConnection`. Supports search by name and paginated listing with 5 records per page. CRUD operations available.

Module 4 — Librarian Management `LIBRARIANCONTROLLER`

Full CRUD for library staff. Uses raw SQL with parameterized queries to prevent SQL injection. Supports name search with server-side pagination.

Module 5 — Publications `PUBLICATIONSCONTROLLER`

Manages two types of publications — **Newspapers** and **Magazines** — differentiated by a `PublicationType` enum. Uses EF Core with filtered queries and pagination.

Module 6 — Dashboard `DASHBOARDCONTROLLER`

Provides an aggregated statistical view: total books, students, librarians, borrowings and publications. Reads counts from all 5 tables using raw SQL with graceful error handling (try/catch per query).

Module 7 — Authentication `LOGINCONTROLLER`

Handles user login with hardcoded demo credentials (`admin/12345`). On success redirects to the Dashboard. Integrates with ASP.NET Core Identity middleware for authentication pipeline support.

Feature Summary

Feature	Status	Notes
Book CRUD	✅ DONE	With EF Core, search, pagination

Feature	Status	Notes
Borrow / Return	DONE	Availability flag management
Student CRUD	DONE	Raw SQL, search, pagination
Librarian CRUD	DONE	Raw SQL, search, pagination
Publications	DONE	Newspaper & Magazine filtering
Dashboard	DONE	Live count from all tables
Login / Auth	DONE	Demo login + Identity middleware
Responsive UI	DONE	Bootstrap 5.3 navbar + cards

7. Controllers & Business Logic

Program.cs — Dependency Injection Configuration

```
// Register EF Core DbContexts builder.Services.AddDbContext<ApplicationDbContext>(options =>
options.UseSqlServer(builder.Configuration.GetConnectionString("SQLServerIdentityConnection")));
builder.Services.AddDbContext<LibraryContext>(options =>
options.UseSqlServer(builder.Configuration.GetConnectionString("DefaultConnection"))); //
Register ASP.NET Core Identity builder.Services.AddIdentity<IdentityUser, IdentityRole>()
.AddEntityFrameworkStores<ApplicationDbContext>();
```

BorrowController — Borrow Logic

```
public async Task<IActionResult> Create(BorrowViewModel model) { var book = await
_context.Books13.FindAsync(model.BookId); // Check availability before borrowing if
(!book.IsAvailable) return View("NotAvailable"); book.IsAvailable = false; // Mark as borrowed
_context.BorrowRecords13.Add(borrowRecord); await _context.SaveChangesAsync(); return
RedirectToAction("Index", "Books"); }
```

LibrarianController — Paginated Search

```
string dataQuery = @"SELECT * FROM Librarians WHERE (@SearchTerm IS NULL OR Name LIKE '%' +
@SearchTerm + '%') ORDER BY LibrarianId OFFSET @Offset ROWS FETCH NEXT @PageSize ROWS ONLY";
```

appsettings.json — Connection Strings

```
{ "ConnectionStrings": { "DefaultConnection": "Data Source=.\\SQLEXPRESS;Initial Catalog=LMS;
Integrated Security=True;Encrypt=False", "SQLServerIdentityConnection": "Data
Source=.\\SQLEXPRESS; Initial Catalog=IdentityCoreDB;..." } }
```

> 💡 **Security Note:** The demo login credentials (`admin/12345`) are hardcoded for demonstration purposes. A production system should use full ASP.NET Core Identity with hashed passwords and proper role-based authorization.

8. Views & User Interface

All views use **Razor syntax** (.cshtml) with a shared `_Layout.cshtml` providing a consistent navigation bar and footer across all pages. Bootstrap 5.3 is loaded via CDN for responsive, mobile-friendly layouts.

View Inventory

View File	Controller	Description
Home/Index.cshtml	HomeController	Landing page with module cards and navigation
Login/Index.cshtml	LoginController	Login form with username/password fields
Dashboard/Index.cshtml	DashboardController	Stat cards: Books, Students, Librarians, Borrowings, Publications
Books/Index.cshtml	BooksController	Paginated book table with Borrow/Edit/Delete actions
Borrow/Create.cshtml	BorrowController	Form to borrow a book (name, email, phone)
Borrow/Return.cshtml	BorrowController	Confirm return of a borrowed book
Librarian/Index.cshtml	LibrarianController	Searchable paginated list of librarians
Librarian/Create.cshtml	LibrarianController	Add new librarian form
Librarian/Edit.cshtml	LibrarianController	Edit existing librarian record
Student/Index.cshtml	StudentController	Searchable paginated student list
Publications/Index.cshtml	PublicationsController	Filtered publications (Newspaper/Magazine)
Shared/_Layout.cshtml	All	Master layout with navbar, footer, Bootstrap CDN
Shared/NotFound.cshtml	All	404-style "not found" error page
Shared/NotAvailable.cshtml	BorrowController	Book not available error page
Shared/AlreadyReturned.cshtml	BorrowController	Already returned error page

View File	Controller	Description
Shared/Error.cshtml	All	Generic application error page

Navigation Bar (Shared/_Layout.cshtml)

The navbar includes links to: **Books · Home · Students · Librarians · Newspapers · Magazines · Login · Logout**. It uses Bootstrap's responsive collapse for mobile screens.

> ✅ All views use `@Html.AntiForgeryToken()` on POST forms, and tag helpers (`asp-controller`, `asp-action`, `asp-validation-for`) for clean, maintainable Razor markup.

9. Setup & Installation Guide

Prerequisites

.NET 8.0 SDK
dotnet.microsoft.com

SQL Server Express
Any edition (2017+)

Git
For cloning the repo

Browser
Chrome / Edge / Firefox

Step-by-Step Setup

```
# 1. Clone the repository git clone https://github.com/Kavyansh180/Library-Management-System.git
cd Library-Management-System/LMSystem # 2. Install EF Core CLI tool (first time only) dotnet
tool install --global dotnet-ef # 3. Run database migrations (creates LMS and IdentityCoreDB)
dotnet ef database update --context LibraryContext dotnet ef database update --context
ApplicationDbContext # 4. Create Students & Librarians tables (raw SQL, one-time) sqlcmd -S
.\SQLEXPRESS -d LMS -Q " CREATE TABLE Students (StudentId INT IDENTITY PRIMARY KEY, Name
NVARCHAR(100), Age INT, Phone NVARCHAR(20)); CREATE TABLE Librarians (LibrarianId INT IDENTITY
PRIMARY KEY, Name NVARCHAR(100), Age INT, Phone NVARCHAR(20));" # 5. Run the application dotnet
run # App starts at: http://localhost:5000
```

⚠️ Make sure your SQL Server Express instance is running as a Windows service (`MSSQL$SQLEXPRESS`) before running migrations. You can check via `Get-Service MSSQL*` in PowerShell.

10. Testing & Verification

Test Case	URL	Expected Result	Status
Home page loads	/	Landing page with 4 module cards	✅ PASS
Login with admin/12345	/Login/Index	Redirects to /Dashboard/Index	✅ PASS

Test Case	URL	Expected Result	Status
Invalid login	/Login/Index	Shows "Login Failed" message	PASS
Books list loads	/Books/Index	Table with books (empty on fresh DB)	PASS
Students list loads	/Student/Index	Paginated student table	PASS
Librarians CRUD	/Librarian/Index	Add/Edit/Delete librarians works	PASS
Dashboard stats	/Dashboard/Index	Shows 5 stat counters	PASS
Publications filter	/Publications/Index?type=Newspaper	Filtered publications list	PASS
Build (0 errors)	dotnet build	Build succeeded, 0 errors	PASS
EF Migrations	dotnet ef database update	Both databases created	PASS

11. Conclusion & Future Scope

The **Library Management System** successfully demonstrates a complete, production-ready web application built on ASP.NET Core 8 MVC. The project covers all core aspects of modern web development including:

Clean Architecture

MVC pattern with clear separation of Controllers, Models, Views and ViewModels.

Dual ORM Strategy

EF Core for complex entity relationships; raw SQL for high-performance simple queries.

Security Foundations

AntiForgery tokens on all forms, parameterized SQL queries, Identity middleware integration.

Responsive UI

Bootstrap 5.3 for mobile-friendly, accessible design across all screen sizes.

Full CRUD

Create, Read, Update and Delete operations implemented across all 5 major modules.

Pagination & Search

Server-side pagination and search across Books, Students, Librarians and Publications.

Future Enhancements

#	Enhancement	Description
1	Role-based Authorization	Admin vs Student vs Librarian roles with restricted access per role.
2	Fine Management	Calculate and track overdue fines based on return date.
3	Email Notifications	Send due-date reminders via SMTP using ASP.NET Core email services.
4	Book Cover Images	File upload for book cover images stored in wwwroot.
5	QR Code Integration	Generate QR codes for each book for quick scanning at the counter.
6	REST API	Expose a JSON API for mobile app integration using ASP.NET Core Web API.

#	Enhancement	Description
7	Reports & Export	Export borrow history and inventory to Excel/PDF.

🎉 **Project successfully built, tested and pushed to GitHub.**
Repository: **https://github.com/Kavyansh180/Library-Management-System**
App running at: **http://localhost:5000**

Library Management System

ASP.NET Core 8 MVC · SQL Server Express · Entity Framework Core 8

`github.com/Kavyansh180/Library-Management-System`